NTNU

Kunnskap for en bedre verden

Department of Computer Science

IT3212 - Data Powered Software

Assignment 2

Authors:
Elling Bakås
Fredrik Nymoen
Johan Fredrik Wilvang
Mats Aleksander Larsson

XX.10.2026

Table of Contents

List of Figures iii

List of Tables iii

1 Introduction 1

2 Fourier Transform 1

2.1 Low-pass filters . . . 1

2.2 High-pass filters . . . 2

3 Principal Component Analysis 2

3.1 PCA Implementation . . . 2

3.2 Reconstruction of Images . . . 2

3.3 Experimentation . . . 3

3.4 Visual Analysis . . . 4

3.5 Error Analysis . . . 5

4 Image Processing 6

4.1 HOG Features . . . 6

4.1.1 Implementation and Images . . . 6

4.1.2 Visualizing the Five Scenes . . . 7

4.1.3 Varying Cell Size, Block Size and Bins . . . 8

4.2 Local Binary Patterns . . . 9

4.2.1 Implementation, LBP Image and Histogram . . . 9

4.2.2 What the Histogram Represents . . . 10

4.2.3 Comparing the Four Histograms . . . 11

4.3 Blob Detection . . . 12

4.3.1 Apply blob detection to the dataset . . . 12

4.3.2 Visualize the detected blobs . . . 13

4.3.3 Calculate and display counts, sizes, and positions . . . 13

4.3.4 Evaluate the effect of algorithm parameters . . . 14

4.4 Contour Detection . . . 16

4.4.1 Apply contour detection to the same dataset . . . 16

4.4.2 Visualize contours using different colors . . . 16

4.4.3 Calculate counts, areas, and perimeters . . . 17

4.4.4 Compare blob and contour detection . . . 17

4.4.5 Discuss advantages and limitations 17
4.4.6 Analyze threshold and filter-size effects 18
4.4.7 Give examples of suitable applications 19

List of Figures

1 Grayscale input image and its corresponding 2D Fourier transform. The zero-frequency component is shifted to the center, with low spatial frequencies located near the center and higher spatial frequencies farther away. 1

2 Effect of different low-pass filter cutoff ratios. The top row shows the resulting images, while the bottom row shows the corresponding low-pass masks in the frequency domain. 1

3 Effect of different high-pass filter cutoff ratios. The top row shows the resulting images, while the bottom row shows the corresponding high-pass masks in the frequency domain. 2

4 Resized grayscale inputs (top) and PCA reconstructions using $k = 50$ (bottom), shown with the same grayscale range. 3

5 Cumulative explained variance. Markers show the smallest number of components needed to reach 80%, 90%, 95% and 99%. 4

6 Original resized images and reconstructions for different values of k, using the same grayscale range. 4

7 MSE versus (a) component count and (b) relative storage, including the components and mean image. 5

8 Image, gradient magnitude and HOG descriptor (8×8 cells, 9 bins). Each segment lies along the edge, perpendicular to the gradient binned in Figure 9. 7

9 HOG energy per orientation bin, where 0° is a vertical edge. 8

10 The sedan's descriptor at 4, 9 and 18 orientation bins. At four the stars collapse to crosses, and at 18 neighboring segments nearly coincide. 9

11 The four images, their LBP codes as grayscale levels, and the number of bits set. . 10

12 LBP histograms, linear (top) and logarithmic (bottom). 11

13 Originals and baseline detections for two display examples, at processed resolution. Yellow circles mark bright blobs and cyan circles dark blobs. Only pickup IDs 1–3 are printed for readability; every detected circle is shown. 13

14 Threshold sensitivity for pickup (top) and sedan (bottom). Threshold increases from left to right; every other parameter remains at baseline. Panel counts concern the individual example images. 15

15 Contour baseline on `sedan/PIC_0.jpg`: original, binary mask, and 18 colored, numbered closed contours. Contour 3 follows a large rear-window region; smaller loops include vehicle details and background patches. 16

16 Contour parameter experiments on `pickup/PIC_0.jpg`. Baseline and lower threshold both retain 14 contours, despite visibly different boundaries. Colors are assigned independently per panel and do not track the same contour across settings. 18

List of Tables

1 Explained variance, mean reconstruction error and theoretical storage relative to the input matrix. 5

2 One HOG parameter varied at a time, on 610 vehicle images, 122 per class. Chance is 0.200. 8

3 LBP histogram statistics. Entropy is in bits, and Uniform and Code 255 are percentages of pixels. 11

4 Blob-detector parameters and baseline values. Scale and smoothing values are measured in processed-image pixels. 12

5 Blob baseline for the ten display images. Mean radius is measured in processed-image pixels. Full statistics cover all 1,310 images. 14

6 First three numbered blobs in `pickup/PIC_0.jpg`. Coordinates and estimated radii are in processed-image pixels; all detections are retained in the CSV export. 14

7 Blob parameter experiments on all 1,310 images. Each variant changes only the specified parameter from baseline. Mean radius is weighted over individual blobs, in processed pixels. 14

8 Contour baseline for all ten analyzed images. Area and perimeter means are calculated over retained loops within each image. 17

9 Contour experiments on the same ten images. Means are weighted over retained loops; area is in square pixels and perimeter in pixels. Other parameters remain at baseline. 18

1 Introduction

2 Fourier Transform

The 2D Fourier transform is strongest at the center, indicating dominant low-frequency components from the image's overall structure and brightness. The rays extending from the center represent higher-frequency components caused by edges and directional features.

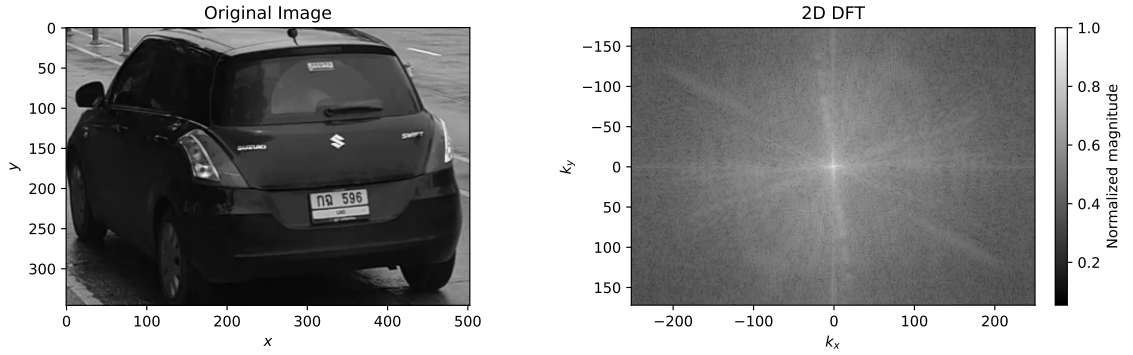


Figure 1: Grayscale input image and its corresponding 2D Fourier transform. The zero-frequency component is shifted to the center, with low spatial frequencies located near the center and higher spatial frequencies farther away.

2.1 Low-pass filters

A low-pass filter removes frequency components above a specified cutoff frequency. This can be used to reduce high-frequency noise in an image. Figure 2 shows the effect of applying low-pass filters with different cutoff ratios to the original image.

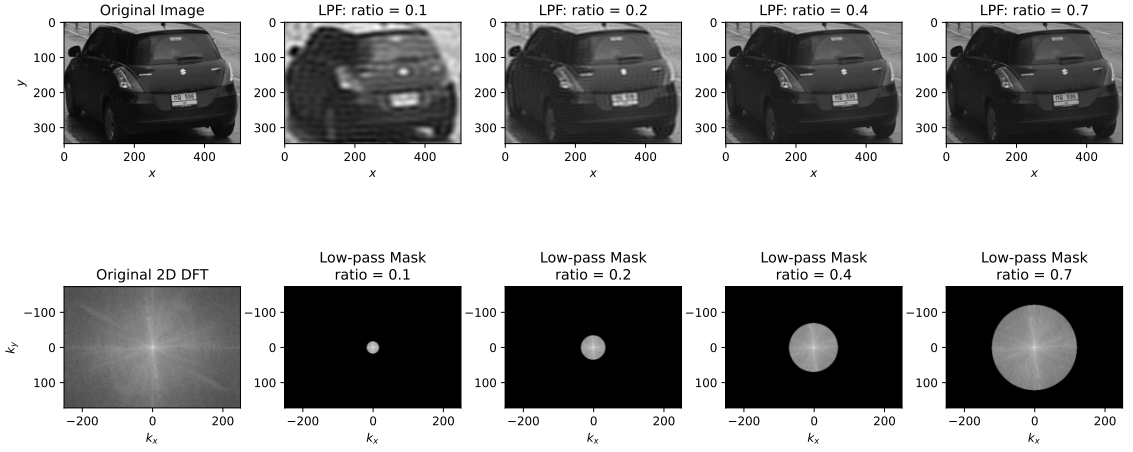


Figure 2: Effect of different low-pass filter cutoff ratios. The top row shows the resulting images, while the bottom row shows the corresponding low-pass masks in the frequency domain.

The results show that the image retains most of its visual information for cutoff ratios of 0.4 and above. In contrast, cutoff ratios of 0.2 and below result in noticeable blurring due to the removal of high-frequency components, which contain information about edges and fine details. Nevertheless, the overall structure and shape of the vehicle remain visible even at the lowest cutoff ratio.

2.2 High-pass filters

A high-pass filter removes low-frequency components while retaining the high-frequency components of an image. Since edges and fine details correspond to rapid changes in intensity, high-pass filtering can be used to emphasize these features. Figure 3 shows the effect of applying high-pass filters with different cutoff ratios to the original image.

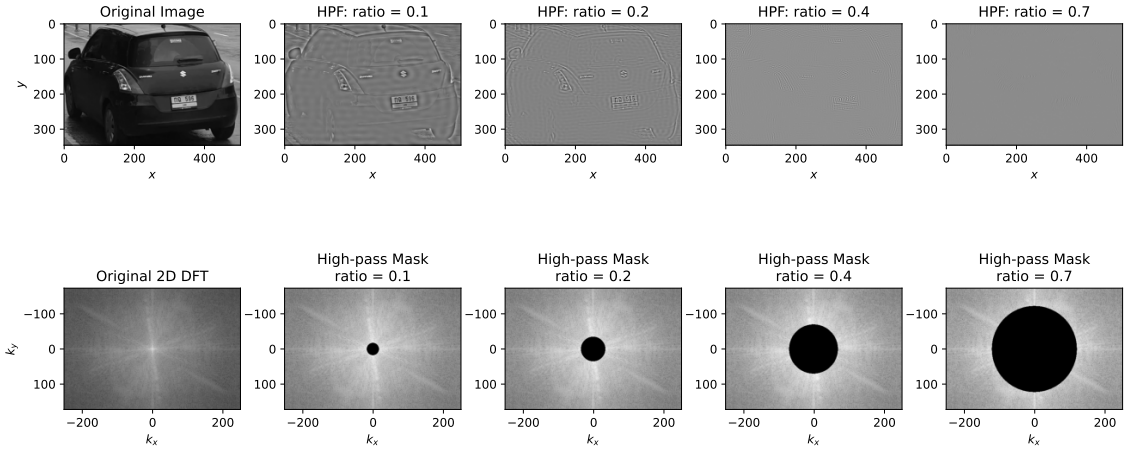


Figure 3: Effect of different high-pass filter cutoff ratios. The top row shows the resulting images, while the bottom row shows the corresponding high-pass masks in the frequency domain.

The results show that the image retains most

3 Principal Component Analysis

3.1 PCA Implementation

We resized the grayscale images to 64×64 pixels and scaled their values to $[0, 1]$. This gave all images the same dimensions and kept the PCA computation cheap, since the covariance matrix grows quadratically with the number of pixels per image. Each image was flattened into a row of 4096 pixel values so that the dataset formed a matrix with one image per row and one pixel position per column. The mean image was subtracted before PCA to centre the data, allowing the principal components to describe variation around the average image. Our function calculated the covariance matrix and its eigenvalues and eigenvectors to find the components that explained the most variation. The eigenvectors were ranked by their corresponding eigenvalues in descending order, and the first k were retained as the principal components. Each centred image was projected onto the selected principal components using matrix multiplication, producing k values instead of 4096.

3.2 Reconstruction of Images

We reconstructed the images by multiplying their PCA representations by the transpose of the retained component matrix and adding the mean image back. Each reconstructed row was then reshaped into a 64×64 image. Information in the discarded components could not be recovered. The comparisons use the resized grayscale inputs, so they do not measure detail already lost during preprocessing.

Figure 4 compares three resized grayscale inputs with their reconstructions using $k = 50$. The overall shapes and light and dark areas are retained, but edges and small details are blurred or lost.



Figure 4: Resized grayscale inputs (top) and PCA reconstructions using $k = 50$ (bottom), shown with the same grayscale range.

3.3 Experimentation

We tested 5, 20, 50, 100, 242 and 500 components using the same PCA basis. For each setting, we retained the first k components and reconstructed the images, so only the number of retained components changed.

Figure 5 shows the cumulative explained variance. The first components capture much of the variation, while later components provide smaller gains. For example, 100 components retain 80.2% of the variance, 242 retain 90.0%, and 500 retain 96.3%.

We chose $k = 100$ as a compromise between a compact representation and image quality. Each image is represented by 100 values instead of 4096, while retaining 80.2% of the variance. Including the shared components and mean image, the theoretical storage is 10.2% of the input matrix, assuming equal precision per value. This gives a storage reduction of 89.8% and fewer input features for later models. Using 242 components retains another 9.8 percentage points of variance, but increases storage to 24.5%.

The disadvantage is that some information is lost, and small details remain blurred at $k = 100$. The PCA values are also harder to interpret directly because each component combines many pixel positions rather than describing a clear feature such as a wheel or window. High explained variance therefore does not guarantee that important vehicle details are preserved. We assess this further through the image comparisons and error analysis.

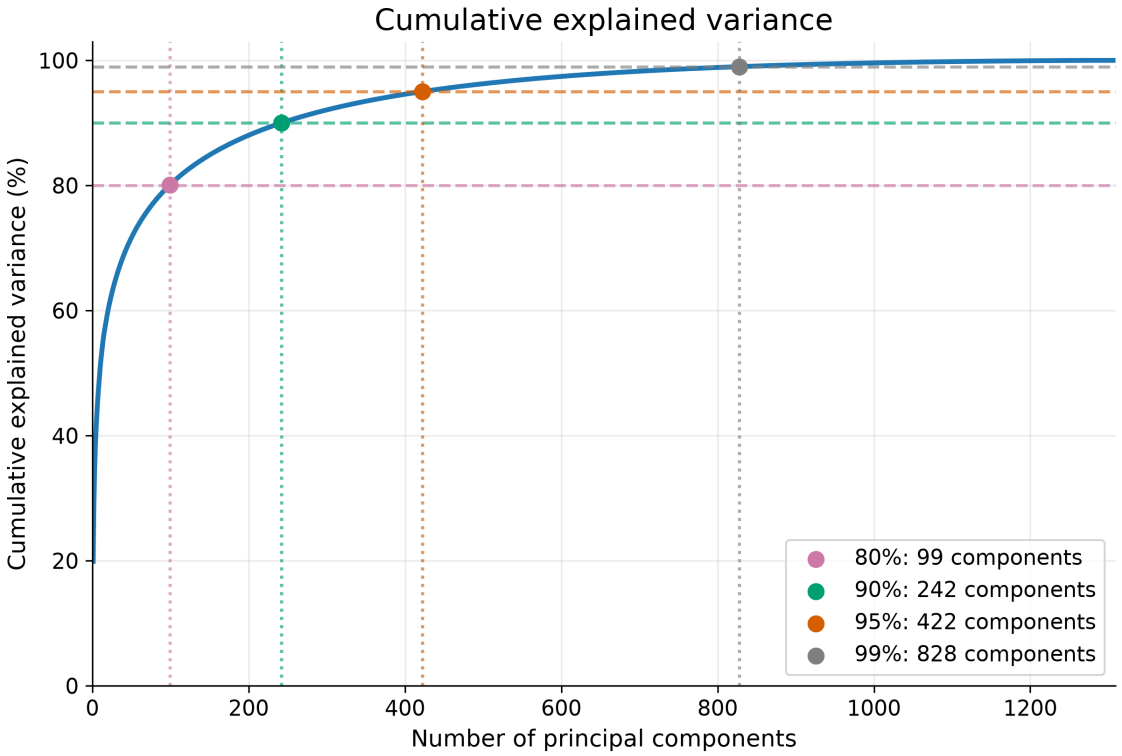


Figure 5: Cumulative explained variance. Markers show the smallest number of components needed to reach 80%, 90%, 95% and 99%.

3.4 Visual Analysis

Figure 6 compares the same three resized grayscale inputs with reconstructions for selected values of k.

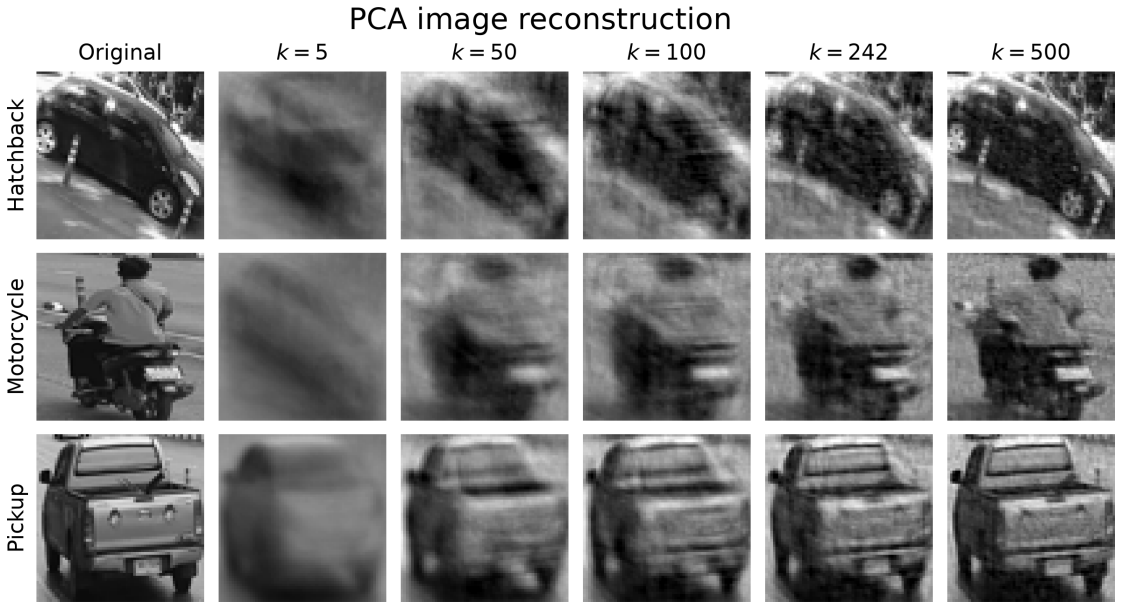


Figure 6: Original resized images and reconstructions for different values of k, using the same grayscale range.

With five components, only light and dark areas and some rough shapes remain. The hatchback, motorcycle and rider are difficult to distinguish, while it is possible to make out the shape of the pickup. At 50 and 100 components, the vehicles shapes becomes clearer, but edges and small details are still blurred. At 242 and 500, more detail returns, it becomes easier to distinguish the rider from the motorcycle and the wheels on the hatchback. However, the hatchback remains noticeably blurred at 242.

The improvement differs between images. High explained variance does not guarantee that every image retains the same level of detail. PCA captures variation in pixel values, including backgrounds and lighting, rather than specifically preserving vehicle features. The variance curve should therefore be interpreted together with the reconstructed images.

3.5 Error Analysis

We used mean squared error (MSE), the average squared difference in pixel values, to compare the resized inputs and the reconstructions. Error was measured before clipping the reconstructed values for display. Lower MSE means less error, but does not fully describe visual quality. Table 1 gives the results for all tested settings.

Table 1: Explained variance, mean reconstruction error and theoretical storage relative to the input matrix.

k	Variance (%)	MSE	Storage (%)
5	41.6	0.027834	0.6
20	60.0	0.019061	2.1
50	71.6	0.013501	5.1
100	80.2	0.009437	10.2
242	90.0	0.004761	24.5
500	96.3	0.001779	50.5

Storage estimates include the reduced image values, shared components and mean image. Counting only the values per image would overstate the compression. The estimates assume equal precision per value and compare matrix sizes rather than JPEG file sizes.

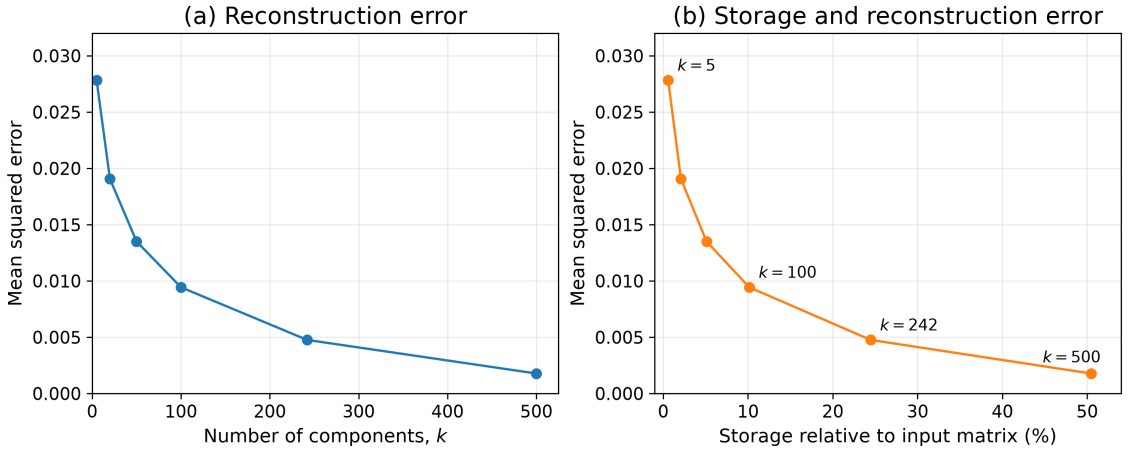


Figure 7: MSE versus (a) component count and (b) relative storage, including the components and mean image.

Figure 7 shows the trade-off: more components give lower error but require more storage. Moving from 100 to 242 roughly halves MSE, while storage increases from 10.2% to 24.5%. At 500, error falls further, but storage reaches 50.5% of the input matrix.

If small storage is the main goal, keeping 100 components is a reasonable compromise. It retains 80.2% of the variance using about 10.2% of the original matrix's storage, while accepting visible blurring. For finer detail, 242 or 500 components are better choices. The preferred setting therefore depends on how much detail the application needs.

These errors are averages, so they can hide differences between individual images. PCA was also evaluated on the images used to calculate its components; performance on new images was not tested.

4 Image Processing

4.1 HOG Features

4.1.1 Implementation and Images

We computed the HOG features with the `hog` function of scikit-image, using the settings of Dalal and Triggs (2005Histograms of Oriented Gradients for Human Detection): the unsmoothed $[-1, 0, 1]$ gradient, nine unsigned orientation bins per 8×8 cell weighted by magnitude, and overlapping 2×2 blocks under L2-Hys, giving 36 values per block and 8,100 in all at 128×128. The parameter study used our own implementation, which matches the library exactly when the vote is not interpolated. As simple and complex scenes we took a horse silhouette from the scikit-image samples and four images of increasing clutter from the vehicle-type dataset (Figure 8). All images were converted to grayscale and squeezed to a square, since HOG needs one descriptor length. This distorts proportions by up to a factor of 1.5 and leaves the portrait motorcycle crops separable on shape alone.

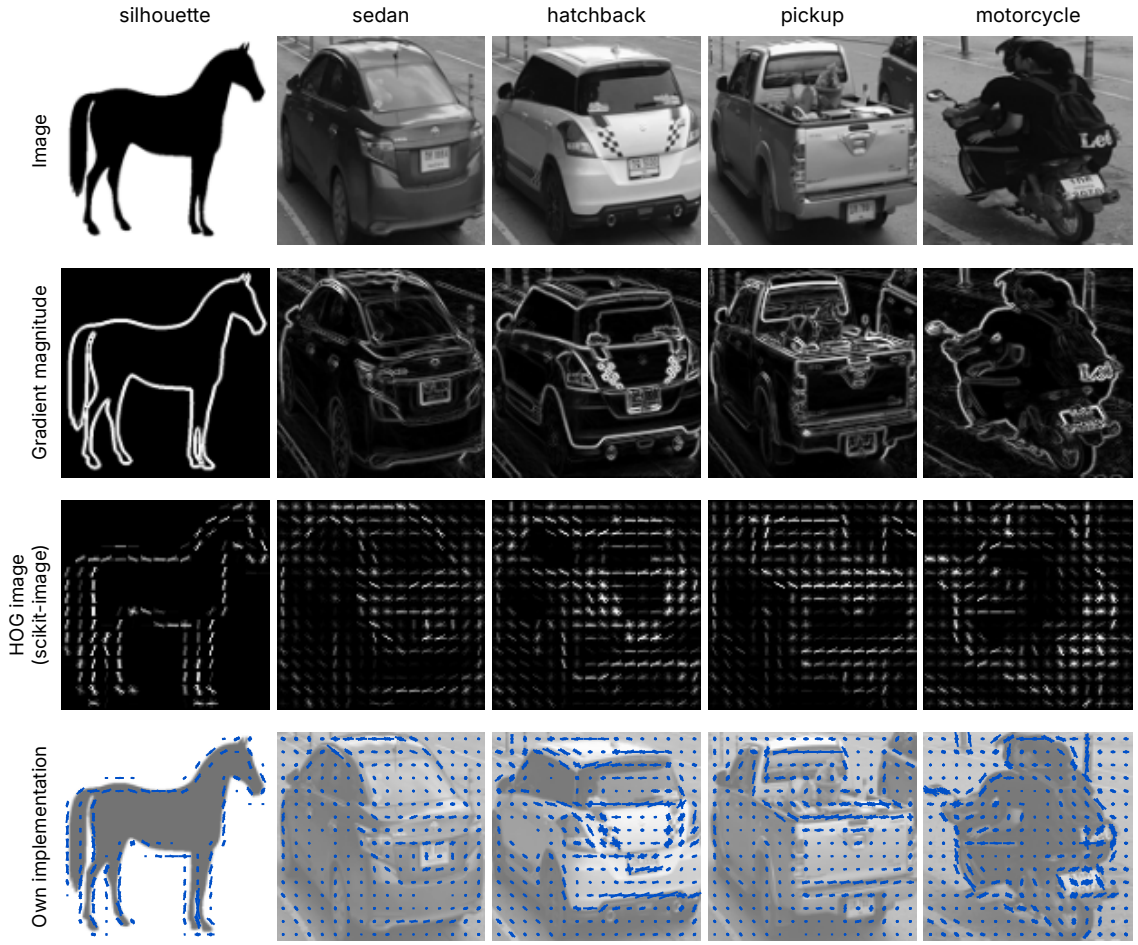


Figure 8: Image, gradient magnitude and HOG descriptor (8×8 cells, 9 bins). Each segment lies along the edge, perpendicular to the gradient binned in Figure 9.

4.1.2 Visualizing the Five Scenes

The silhouette's descriptor traces the contour and leaves the rest empty. Only 13.1% of its pixels exceed a gradient magnitude of 0.08, and 81.1% of its values fall below 0.01. For the vehicles the edge share rises from 30.7% (sedan) to 40.7% (pickup) and the near-zero share drops to 7 to 18%, because a decal, a load and a second vehicle add edges, while block normalization lifts faint road texture to unit norm. HOG has no segmentation step.

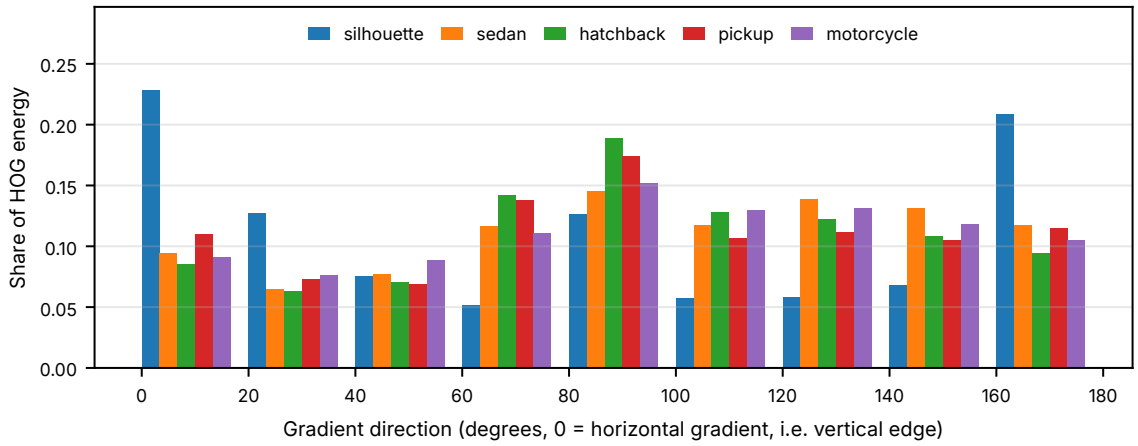


Figure 9: HOG energy per orientation bin, where 0° is a vertical edge.

Figure 9 bins the gradient direction, perpendicular to the edge, so 0° is a vertical edge. The silhouette peaks there and every vehicle at 90°, with no bin separating two vehicles by more than five points. What separates them is *where* the orientations fall, and the full descriptors are 0.69 to 0.72 alike among the cars but 0.57 between a car and the motorcycle.

Table 2: One HOG parameter varied at a time, on 610 vehicle images, 122 per class. Chance is 0.200.

Parameter	Value	Length	Gap	Accuracy	McNemar p
Baseline	8, 2, 9	8,100	0.032	0.915	–
Cell size	4	34,596	0.024	0.890	0.014
	16	1,764	0.035	0.900	0.163
	32	324	0.023	0.890	0.040
Block size	1	2,304	0.019	0.893	0.053
	3	15,876	0.040	0.920	0.629
Bins	4	3,600	0.019	0.900	0.093
	18	16,200	0.036	0.916	1.000

4.1.3 Varying Cell Size, Block Size and Bins

Table 2 scores each setting by the *cosine gap*, mean within-class minus mean between-class similarity, and by leave-one-out 1-NN accuracy on 610 images, 122 per class, the largest balanced subset the smallest class allows. Neither is a classifier. The gap is a property of the descriptors, while the accuracy also rises with sample density, so only comparisons at a fixed sample size mean anything. Both extremes of cell size are significantly worse than the 8×8 baseline, at $p = 0.014$ and $p = 0.040$. Small cells ask whether an edge sits at exactly this position, which shifts with camera distance, while large cells merge the roofline and the rear window. Between 8 and 16 the accuracy is indistinguishable ($p = 0.163$), but 16×16 has the higher gap at a fifth of the length, so we chose it. Larger blocks and more bins raise the gap further, yet cost double the length for no change in accuracy. Figure 10 shows what the extra bins buy.

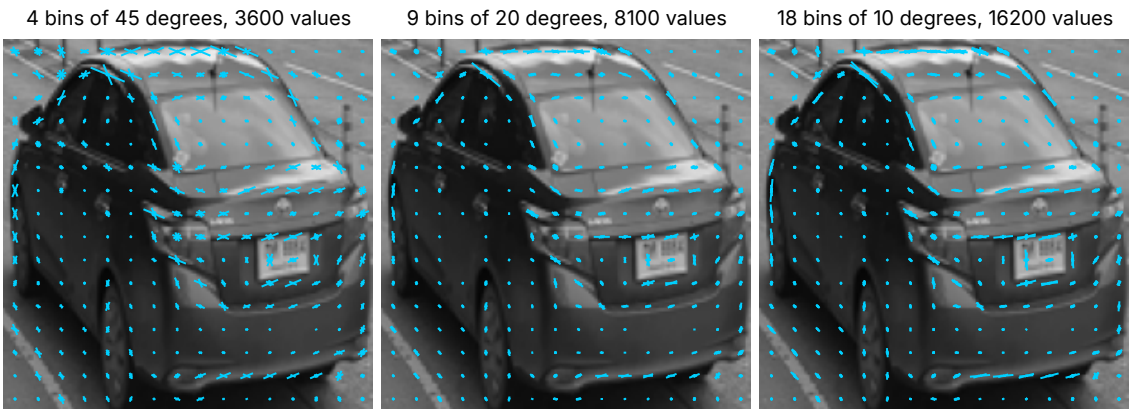


Figure 10: The sedan's descriptor at 4, 9 and 18 orientation bins. At four the stars collapse to crosses, and at 18 neighboring segments nearly coincide.

4.2 Local Binary Patterns

4.2.1 Implementation, LBP Image and Histogram

LBP describes the texture around a pixel by recording which of its eight neighbors are at least as bright as the center. Our own function visits them clockwise from the top left and writes the eight results as one byte (Ojala et al. 2002Multiresolution Gray-Scale and Rotation Invariant Texture Classification with Local Binary Patterns),

$$\mathrm{LBP}(x,y) = \sum_{p=0}^{7} s(g_p - g_c)\, 2^p, \qquad s(t) = \begin{cases} 1, & t \geq 0 \\ 0, & t < 0 \end{cases} \tag{1}$$

where g_c is the center and g_p neighbor p. It copies the border, so the output is a same-size image of codes. A second function counts the 256 codes one by one, without `np.histogram` or `np.bincount`, and normalizes the result. Both were checked against all 256 neighborhoods. We applied them to a natural scene (`camera`), a texture (`brick`), a face (`astronaut`) and a sedan from the vehicle-type dataset, all grayscale at 128×128 (Figure 11).

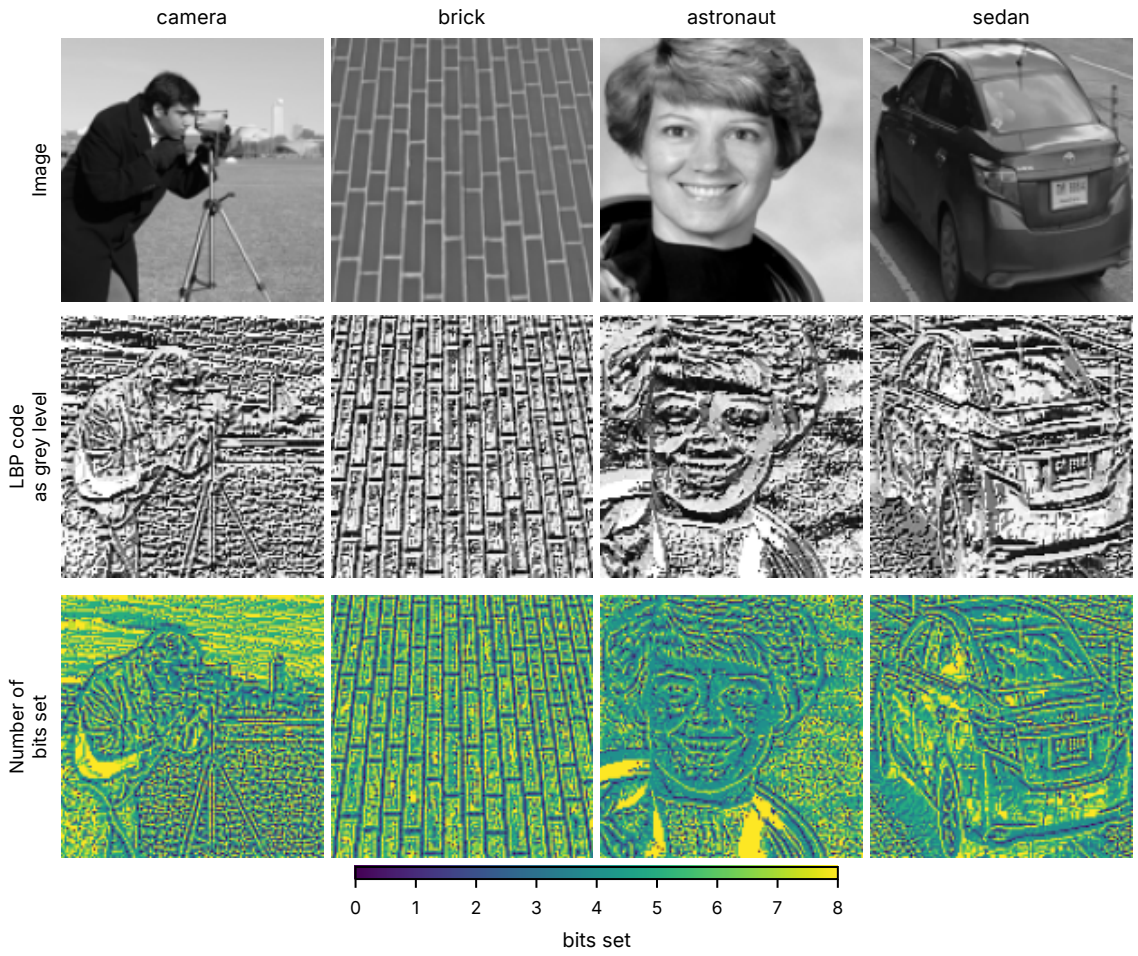


Figure 11: The four images, their LBP codes as grayscale levels, and the number of bits set.

4.2.2 What the Histogram Represents

The histogram discards where each pattern occurred and keeps the micro-texture (Figure 12). Code 255 means no neighbor is darker. Since ties count as one it comes mostly from equal neighbors, so it measures how smooth the image is at pixel scale. Code 0 is a strict local maximum. A code with one run of ones, such as $00001111_2 = 15$, says one side of the neighborhood is brighter, and which of those dominate gives the edge orientation. The 58 *uniform* codes, those with at most two transitions, are 23% of the alphabet but cover 80 to 87% of the pixels, hence the usual reduction to 59 bins. We did not use it, because the descriptor is short enough here that a 4.3× compression buys nothing.

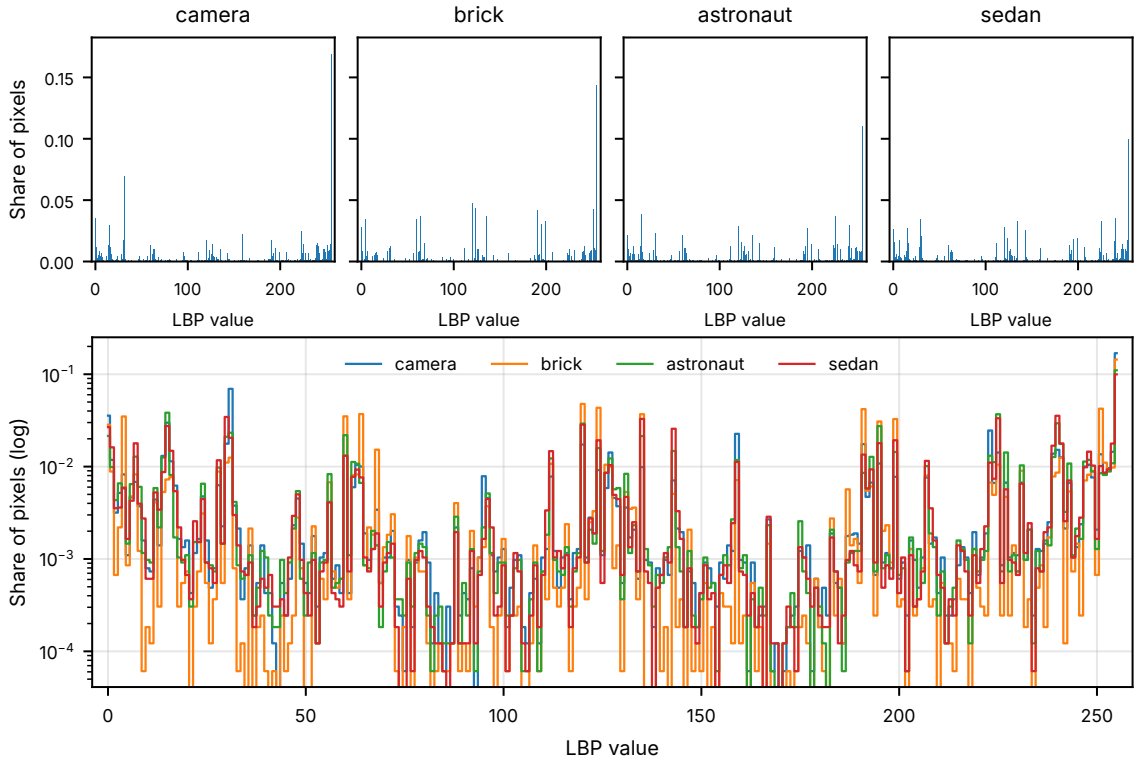


Figure 12: LBP histograms, linear (top) and logarithmic (bottom).

Table 3: LBP histogram statistics. Entropy is in bits, and Uniform and Code 255 are percentages of pixels.

Image	Entropy	Uniform	Code 255	Top codes	What the histogram says
Natural scene (camera)	6.17	80.4	16.9	255, 31, 0	Mixed structure plus one large smooth region, the sky
Texture (brick)	5.75	86.7	14.4	255, 120, 124	One repeating pattern, so few codes dominate
Face (astronaut)	6.38	83.3	11.0	255, 15, 225	Smooth shading, edges in several orientations
Vehicle (sedan)	6.35	82.8	10.0	255, 240, 30	Similar to the face, smooth shading and edges

4.2.3 Comparing the Four Histograms

The four differ as their textures do (Table 3), with the reservation that we used one image of each kind. Brick has the lowest entropy and the highest uniform share, because a repeating texture produces the same few patterns everywhere, here the one-sided runs along the brick gaps. Under the χ^2 distance it is the outlier, 0.15 to 0.16 from the others. The natural scene has the largest share of code 255, and its second code is sky rather than an edge, covering 12.9% of the upper half of that image against 1.0% of the lower, because LBP gives a slow gradient the same code as a sharp one.

The closest pair is the face and the sedan, at 0.029, though they share no subject. Both are a rounded object shaded smoothly on a plain background, the same surface at 3×3 scale. LBP measures local structure and discards layout, so a global histogram describes texture, not objects.

Finally, Equation 1 uses only the sign of each difference, so any strictly increasing change of the grayscale values leaves every code identical: a linear change, a gamma correction and a logarithm each left 100% of the codes unchanged at $\chi^2 = 0$. That is one important advantage of LBP over HOG, where block normalization gives only approximate illumination invariance.

4.3 Blob Detection

4.3.1 Apply blob detection to the dataset

A multiscale Laplacian-of-Gaussian (LoG) detector was applied to all 1,310 images to locate bright and dark regions and estimate their spatial scale. Images were EXIF-oriented, resized to a maximum side of 512 pixels without enlargement, and converted to grayscale intensities in $[0, 1]$. Aspect ratios were preserved; all subsequent measurements refer to processed-image pixels.

The detector smooths the image and measures local intensity curvature at several scales. For grayscale image I, this gives the response

$$\boxed{R_\sigma(x, y) = -\sigma^2 \nabla^2 (G_\sigma * I)(x, y)}. \tag{2}$$

In Equation (2), (x, y) is pixel position, G_σ is a Gaussian filter with standard deviation σ, and $*$ denotes convolution. The Laplacian ∇^2 sums the second spatial derivatives. Small values of σ examine fine structures; larger values examine broader regions. Multiplication by σ^2 normalizes the response across scales, while the minus sign assigns positive responses to bright centers and negative responses to dark centers.

Candidate centers are local maxima of the absolute response within three rows, three columns, and three neighboring scale levels. A response threshold τ excludes weak candidates. Each remaining candidate receives radius $r = \sqrt{2}\sigma$, an estimate of feature scale rather than an object boundary. Circles crossing the frame are excluded. Candidates are then processed strongest first, discarding those whose overlap with a retained circle exceeds a specified fraction of the smaller circle's area.

These decisions require a common configuration before results can be compared. Table 4 defines that reference configuration, called the baseline, and the role of each parameter. The values were selected through visual inspection of the display examples described below, rather than optimization against annotated data. Additional pre-smoothing is disabled because Gaussian smoothing is already part of LoG; its effect is tested later.

Table 4: Blob-detector parameters and baseline values. Scale and smoothing values are measured in processed-image pixels.

Parameter	Baseline	Function
Minimum scale, $\sigma_{\min}$	4	Smallest Gaussian scale examined; sets the lower feature-size limit.
Maximum scale, $\sigma_{\max}$	16	Largest Gaussian scale examined; sets the upper feature-size limit.
Number of scales	12	Number of filter scales between the limits, spaced with a constant ratio.
Response threshold, τ	0.15	Minimum absolute LoG response required for a candidate.
Overlap limit	0.5	Maximum permitted intersection as a fraction of the smaller circle's area.
Pre-smoothing, σ_{pre}	0	Gaussian smoothing before LoG filtering; zero disables this additional step.

SciPy supplies the Gaussian derivatives and neighborhood maxima; candidate selection and suppression are implemented explicitly. Synthetic checks verified centers, polarity, approximate scales, and no detections on a constant image.

4.3.2 Visualize the detected blobs

All images were processed and received numbered overlays. For presentation, ten fixed examples were selected by sorting filenames alphabetically within each vehicle class and taking the first two from each of the five classes. This provides reproducible examples from every class without reproducing all 1,310 outputs. It is not a random or representative sample, and it does not limit the full-dataset blob calculations.

Figure 13 shows two of these examples: a pickup with repeated trim features and a sedan with broader window and body regions. They were selected to examine how local structure affects the detections. Yellow circles mark bright responses and cyan circles dark responses.



Figure 13: Originals and baseline detections for two display examples, at processed resolution. Yellow circles mark bright blobs and cyan circles dark blobs. Only pickup IDs 1–3 are printed for readability; every detected circle is shown.

The pickup has repeated responses along the window edges and bumper. The sedan has a large dark response on part of its rear window, alongside smaller responses on reflections and other details. Background regions are marked in both images. The output therefore describes local contrast, not vehicle instances or complete object outlines.

4.3.3 Calculate and display counts, sizes, and positions

Across the full dataset, the baseline produced 85,074 blobs with a mean radius of 7.40 pixels. Counts range from 3 to 209 per image; the median is 63, and the middle 50% of images have 42–83 detections. Thus, the total combines images with substantially different numbers of local responses.

Table 5 reports counts and mean radii for the ten fixed display examples. These are excerpts from the complete per-image results.

Table 5: Blob baseline for the ten display images. Mean radius is measured in processed-image pixels. Full statistics cover all 1,310 images.

Image	Blobs	Mean radius (px)
hatchback/PIC_0.jpg	33	7.22
hatchback/PIC_1.jpg	85	7.70
motorcycle/PIC_0.jpg	83	7.23
motorcycle/PIC_10.jpg	19	6.10
pickup/PIC_0.jpg	110	6.96
pickup/PIC_1.jpg	106	6.74
sedan/PIC_0.jpg	27	8.14
sedan/PIC_1.jpg	70	9.68
suv/PIC_0.jpg	107	7.12
suv/PIC_1.jpg	55	8.21

To connect these summaries to individual detections, Table 6 gives the first three pickup blobs, whose IDs are marked in the overlay. Coordinates begin at the top-left corner, with x increasing rightward and y downward. Radius r determines the plotted circle; its diameter and approximate area are $2r$ and πr^2. These are image-feature measurements, not physical vehicle dimensions.

Table 6: First three numbered blobs in `pickup/PIC_0.jpg`. Coordinates and estimated radii are in processed-image pixels; all detections are retained in the CSV export.

ID	x	y	r	Polarity
1	15	60	9.36	dark
2	52	186	10.62	bright
3	91	13	7.28	bright

CSV exports retain all 1,310 image summaries and 85,074 individual blob records. The measurements describe where the detector responds and at what scale. Without annotated blobs, they do not establish detection accuracy.

4.3.4 Evaluate the effect of algorithm parameters

The baseline results depend on which scales and response strengths are retained. To distinguish these effects, six variants were evaluated on all 1,310 images, each changing one baseline parameter. Table 7 summarizes their counts and blob-weighted mean radii.

Table 7: Blob parameter experiments on all 1,310 images. Each variant changes only the specified parameter from baseline. Mean radius is weighted over individual blobs, in processed pixels.

Setting	Value	Total blobs	Mean radius
Baseline	$\tau = 0.15,\ \sigma = 4$–16	85,074	7.40
Lower threshold	$\tau = 0.075$	198,896	6.88
Higher threshold	$\tau = 0.225$	25,854	7.86
Larger minimum	$\sigma_{\min} = 7$	35,726	11.93
Larger maximum	$\sigma_{\max} = 30$	84,213	7.60
Stricter overlap	Overlap limit $= 0.2$	77,302	7.36
Extra smoothing	Pre-smoothing $\sigma_{\text{pre}} = 2$	44,825	9.42

Response threshold. Lowering τ to 0.075 increased the total to 198,896 (134% above baseline), while raising it to 0.225 reduced it to 25,854 (70% below baseline). The direction of change holds for every image, not only for a few high-count examples. Mean radius rises from 6.88 to 7.86

pixels between these settings, indicating a change in the size distribution as well as the number of retained responses.

Counts do not reveal which structures disappear. Figure 14 therefore compares the same pickup and sedan at three thresholds, holding all other parameters fixed. Using the same images separates threshold effects from differences in image content.

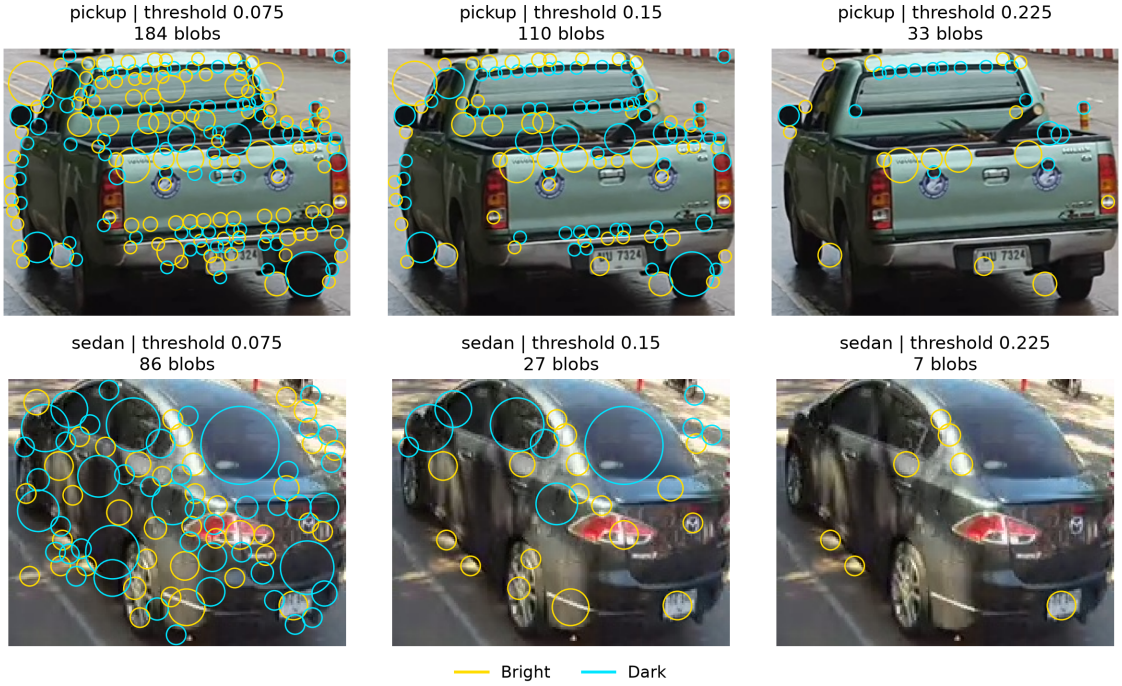


Figure 14: Threshold sensitivity for pickup (top) and sedan (bottom). Threshold increases from left to right; every other parameter remains at baseline. Panel counts concern the individual example images.

The pickup counts fall from 184 to 110 to 33, while the sedan counts fall from 86 to 27 to 7. At the high threshold, repeated pickup window-edge responses remain, but the broad dark sedan window response disappears and several bright reflections remain. A large blob therefore need not have a strong response. The rise in dataset-wide mean radius is a distributional result, not a rule for each feature, and fewer detections do not establish better detection.

Scale range. Raising the minimum scale to 7 excludes radii below 9.90 pixels, reducing the total to 35,726 and increasing mean radius to 11.93 pixels. Increasing the maximum scale to 30 permits radii up to 42.43 pixels, but gives a slightly lower count than baseline (84,213). Because the number of scales remains 12, changing either endpoint also changes the intermediate sampling grid. The result therefore combines a changed size range with changed scale sampling; overlap suppression and frame exclusion further affect which candidates survive.

Overlap and pre-smoothing. Reducing the overlap limit from 0.5 to 0.2 leaves 77,302 blobs by rejecting more competing circles, potentially including nearby distinct features. Additional smoothing with $\sigma_{\text{pre}} = 2$ leaves 44,825 blobs with a mean radius of 9.42 pixels. This reduction in count and increase in mean radius are consistent with suppression of fine detail.

The parameters control different aspects of the representation: threshold selects response strength, scale limits select feature size, and overlap suppression removes competing detections. Their suitability therefore depends on the intended feature, such as a compact reflection or a broader dark region. The experiments establish sensitivity to these choices; without a defined target and independent annotations, they do not establish an optimal setting.

4.4 Contour Detection

4.4.1 Apply contour detection to the same dataset

The blob detector represents local contrast with circles. Contour detection was used to examine the boundaries of intensity regions instead, using the same preprocessing. This analysis covers the ten fixed examples introduced in the blob section. The blob baseline was recomputed on these same inputs so that differences between methods are not caused by different image samples. All contour statistics and parameter experiments concern these ten images; the conclusions do not extend to a full-dataset contour evaluation.

The baseline first smooths normalized grayscale intensities with a Gaussian filter ($\sigma = 1$ pixel), then forms the binary mask in Equation (3):

$$M(x, y) = \begin{cases} 1, & (G_1 * I)(x, y) > 0.5, \\ 0, & \text{otherwise.} \end{cases} \tag{3}$$

ContourPy's serial marching-squares engine traces the mask's 0.5 level, interpolating boundary positions between pixel centers. The implementation retains closed loops of area at least 25 square pixels and records excluded open and small paths. Open paths are not artificially closed across the frame. Outer boundaries and boundaries around holes are counted separately; the algorithm does not build an object hierarchy. Consequently, a contour is a boundary of a mask region, not necessarily a complete vehicle outline.

Geometry checks verified a square's known area, perimeter, and centroid, extraction of a binary rectangle, exclusion of a frame-crossing open path, separate inner/outer ring boundaries, and no detections on a constant image.

4.4.2 Visualize contours using different colors

Figure 15 shows the prepared original, thresholded mask, and retained contours. Each contour has a distinct, deterministic hue within its image and an ID linked to its measurements. Colors do not denote vehicle classes or matching objects across different images.



Figure 15: Contour baseline on `sedan/PIC_0.jpg`: original, binary mask, and 18 colored, numbered closed contours. Contour 3 follows a large rear-window region; smaller loops include vehicle details and background patches.

The window loop follows an irregular region, but large dark portions of the car and surroundings merge in the mask. The retained loops therefore describe selected intensity regions rather than a complete vehicle silhouette. This difference between region boundaries and object boundaries also determines how the following measurements should be interpreted.

4.4.3 Calculate counts, areas, and perimeters

For ordered vertices (x_i, y_i) of a closed loop, with vertex $n + 1$ equal to vertex 1, Equation (4) defines the area A and perimeter P:

$$A = \frac{1}{2}\left|\sum_{i=1}^{n}(x_i y_{i+1} - x_{i+1} y_i)\right|, \qquad P = \sum_{i=1}^{n}\sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2}. \tag{4}$$

Area is measured in processed square pixels and perimeter in processed pixels. The baseline retains 158 contours (7–23 per image), with a contour-weighted mean area of 349.7 square pixels and mean perimeter of 95.1 pixels. Table 8 gives every analyzed image's count and mean measurements. Individual loop areas, perimeters, and centroids are exported in `baseline_contours.csv`. Boundary vertices are saved separately in the contour-results directory.

Table 8: Contour baseline for all ten analyzed images. Area and perimeter means are calculated over retained loops within each image.

Image	Contours	Mean area (px^2)	Mean perimeter (px)
hatchback/PIC_0.jpg	7	139.5	55.0
hatchback/PIC_1.jpg	21	289.9	98.3
motorcycle/PIC_0.jpg	19	93.1	46.0
motorcycle/PIC_10.jpg	7	92.1	42.8
pickup/PIC_0.jpg	14	521.2	140.4
pickup/PIC_1.jpg	20	224.3	105.6
sedan/PIC_0.jpg	18	197.8	52.8
sedan/PIC_1.jpg	23	649.6	114.8
suv/PIC_0.jpg	21	479.8	125.9
suv/PIC_1.jpg	8	677.6	135.8

There are also 166 excluded open paths and 436 excluded small or degenerate closed paths. Counts depend on these retention rules as well as image content. Nested loops measure individual enclosed regions; adding their areas would double-count overlaps rather than measure net foreground area.

4.4.4 Compare blob and contour detection

For the same ten images, the two baselines produce 695 blobs and 158 contours. This matched comparison excludes the remaining images from the full-dataset blob analysis. The larger blob count reflects a different feature definition, not greater accuracy. LoG summarizes local contrast using centers and scale estimates; contours follow the irregular boundaries of thresholded regions.

In `pickup/PIC_0.jpg`, there are 110 blobs but 14 contours (Figures 13 and 16). Repeated circles follow window and bumper details, whereas individual contours can describe elongated window and bumper regions. Neither result identifies the whole pickup. A blob's πr^2 is a scale-based circular approximation, whereas contour area measures an enclosed polygon; these are not interchangeable estimates of the same object. Likewise, the contour brightness threshold of 0.5 and LoG response threshold of 0.15 operate on different quantities.

4.4.5 Discuss advantages and limitations

LoG provides compact center-and-scale features and detects both bright and dark regions without choosing one global foreground brightness class. This representation provides approximate location and scale without tracing a boundary. A circle does not describe the outline of an elongated region, and edges, reflections, and background texture can generate multiple detections. The scale range and frame exclusion also restrict what can be found.

Contours preserve irregular boundary geometry and support area and perimeter measurements. The measured boundaries depend on the segmentation: a global brightness threshold can merge

unrelated regions or fragment one object. Discarding open paths removes frame-intersecting boundaries, and separate hole loops are not equivalent to separate objects. Illumination, perspective, occlusion, and resolution affect both approaches. Neither method uses the vehicle labels to recognize objects, and without boundary or blob annotations detection precision and recall cannot be calculated. The ten-image comparison also cannot establish performance across the full dataset.

4.4.6 Analyze threshold and filter-size effects

The dependence on segmentation was examined by varying brightness threshold, smoothing, and minimum retained area. Each variant changes one baseline parameter on all ten images. Table 9 reports the aggregate changes, while Figure 16 shows their effect on the pickup boundaries. The threshold changes which pixels enter the mask; smoothing changes local intensity variation; minimum area filters the resulting loops without changing their boundaries.

Table 9: Contour experiments on the same ten images. Means are weighted over retained loops; area is in square pixels and perimeter in pixels. Other parameters remain at baseline.

Setting	Value	Contours	Mean area	Mean perimeter
Baseline	$t = 0.5,\ \sigma = 1,\ A_{\min} = 25$	158	349.7	95.1
Lower threshold	$t = 0.35$	182	343.5	93.2
Higher threshold	$t = 0.65$	109	278.0	88.7
No smoothing	$\sigma = 0$	230	342.6	101.8
Stronger smoothing	$\sigma = 2$	96	328.6	88.1
Larger minimum area	$A_{\min} = 100$	65	773.4	172.1



Figure 16: Contour parameter experiments on `pickup/PIC_0.jpg`. Baseline and lower threshold both retain 14 contours, despite visibly different boundaries. Colors are assigned independently per panel and do not track the same contour across settings.

Brightness threshold. Thresholds 0.35, 0.5, and 0.65 produce 182, 158, and 109 contours across the sample. The pickup counts are 14, 14, and 11. Lowering the threshold changes the window,

bumper, and plate boundaries without changing the first two counts. At the high threshold, the broad window region is replaced by a narrow highlight. Thus, equal counts can conceal substantial changes in shape and position. Thresholding alters connectivity, so its effect on count is not generally monotonic even though the sample totals decrease here.

Smoothing. Removing smoothing increases retained contours to 230 and mean perimeter from 95.1 to 101.8 pixels. It produces 1,933 discarded small or degenerate loops, compared with 436 at baseline. The pickup has 27 retained contours with visibly more fragmented and jagged boundaries. Increasing σ to 2 reduces the sample total to 96 and mean perimeter to 88.1 pixels. Small details disappear or merge, but the 13 pickup contours are not simply a subset of the baseline: smoothing can also change connectivity.

Minimum area. Increasing the cutoff from 25 to 100 square pixels reduces the total from 158 to 65, while mean retained area rises from 349.7 to 773.4 square pixels. This leaves the mask and surviving boundaries unchanged; it filters measurements rather than improving segmentation. The pickup retains 10 of its 14 baseline loops.

These experiments show a trade-off between retaining fine detail and suppressing fragmented regions. They do not establish an optimal setting; that requires a target definition and annotated evaluation data.

4.4.7 Give examples of suitable applications

Compact contrast features. Candidate lamp details, reflective spots, or dark patches can be represented by a center and approximate scale. Figure 13 illustrates such local responses in the pickup, although a subsequent classifier or validation step would be needed to identify which responses actually belong to a desired vehicle component.

Shape measurements. The sedan window region in Figure 15 has an irregular outline that is not represented by a single circle. When segmentation isolates the intended region, contours permit direct area and perimeter measurements. A controlled background with a brightness range distinct from the object would allow global thresholding to separate the silhouette. The present photographs contain shadows, reflections, and background regions that overlap the vehicle's brightness range.

For whole-vehicle recognition in these scenes, neither baseline establishes one detection or one complete outline per vehicle. The results support choosing a method according to the required representation: local center/scale features or the geometry of adequately segmented boundaries.